

Java Data Structures & Algorithms Cheat Sheet

Arrays

```
int[] arr = new int[5];    // Declaration
int[] arr2 = {1, 2, 3};    // Initialization
arr[0] = 10;               // Access & modify
int len = arr.length;      // Length
```

Linear Search

```
public int linearSearch(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) return i;
    }
    return -1;
}
```

Binary Search (sorted array)

```
public int binarySearch(int[] arr, int target) {
    int left = 0, right = arr.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

Selection Sort

```
public void selectionSort(int[] arr) {
    for (int i = 0; i < arr.length - 1; i++) {
        int min = i;
        for (int j = i + 1; j < arr.length; j++) {
            if (arr[j] < arr[min]) min = j;
        }
    }
}
```

```
    }
    int temp = arr[i];
    arr[i] = arr[min];
    arr[min] = temp;
}
}
```

Bubble Sort

```
public void bubbleSort(int[] arr) {
    for (int i = 0; i < arr.length - 1; i++) {
        for (int j = 0; j < arr.length - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

Stack (LIFO)

```
import java.util.Stack;
```

```
Stack<Integer> stack = new Stack<>();
stack.push(10);    // Add
int top = stack.pop(); // Remove top
boolean empty = stack.isEmpty();
```

Queue (FIFO)

```
import java.util.LinkedList;
import java.util.Queue;
```

```
Queue<Integer> queue = new LinkedList<>();
queue.add(10);    // Enqueue
int front = queue.remove(); // Dequeue
boolean empty = queue.isEmpty();
```